C SWITCH STATEMENT

MULTI-WAY BRANCHING CONTROL

Essential 7-Page Technical Guide

01. Concept & Syntax

The `switch` statement is a multi-way branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of an expression.

```
switch(expression) {
  case value1:
    // code
    break;
  case value2:
    // code
    break;
  default:
    // default code
}
```

It is often more efficient and readable than a long `if-else if` ladder when checking a single variable against multiple fixed values.

02. The Execution Flow

The internal mechanism of a switch statement follows these steps:

1. **Evaluation:** The expression in `switch()` is evaluated once.
2. **Matching:** The result is compared with the values in each `case` label.
3. **Execution:** If a match is found, the code following that case executes.
4. **Termination:** The `break` keyword exits the switch block.
5. **Fallback:** If no match is found, the `default` block runs.

03. The 'break' Keyword

The `break` statement is vital. Without it, execution will "fall through" into the next case regardless of whether that case matches or not.

> **Fall-through:** Sometimes intentional (e.g., grouping multiple cases to perform the same action), but usually a logical bug if forgotten.

```
case 1:
case 2:
  printf("Both 1 and 2 land here!");
  break;
```

04. Mandatory Rules

- **Integers Only:** The switch expression must return an `int` or `char`. It **cannot** be a float or a string.
- **Constant Labels:** Case values must be constant or literal (e.g., `case 5:` or `case 'A':`). You cannot use variables.
- **Unique Cases:** No two case labels can have the same value.
- **Default Position:** Usually placed at the end, but technically can be anywhere.

05. switch vs if-else

Feature	switch	if-else
Logic Type	Equality only	Ranges (>, <, ==)
Performance	Faster (Jump tables)	Slower (Sequential)
Variable	One variable only	Multiple variables

Best Practice: Always include a `default` case to handle unexpected inputs, ensuring program robustness.

06. Quick Summary

- **Cleanliness:** Ideal for menu-driven programs.
- **Break:** Essential to prevent accidental fall-through.
- **Constant:** Labels must be fixed values.
- **Types:** Works with `int` and `char` only.

END OF C SWITCH GUIDE